

OS -- Threads & Multithreading Models

[Target] Target: Explain Thread vs Process in 20s. Name all 3 multithreading models cold.
Read time: 12 minutes

WHAT IS A THREAD?

A thread is a single sequence stream within a process -- the smallest unit of CPU execution.

Threads are called lightweight processes because they share most of the parent process's resources.

A Thread consists of 3 things:

Thread = [Program Counter] + [Register Set] + [Stack Space]

What threads SHARE (from parent process):

- Code section (text)
- Data section (global variables)
- OS resources (open files, signals)
- Heap (dynamic memory)

What threads have INDEPENDENTLY:

- Program counter (each has its own execution point)
- Register set (own CPU registers)
- Stack (own function call stack)

THREAD vs PROCESS -- 4 KEY DIFFERENCES

#

Threads
Processes

1

Threads are not independent of one another -- they share address space
Processes are independent

2

All threads within a process can access every address in the task
Each process has its own separate address space

3

Threads are designed to assist each other
Processes may or may not assist each other

4

If one thread blocks other threads can continue (in KLT)
If one thread blocks may block entire process (in ULT)

Similarities (Threads & Processes):

1. Only one thread/process active at a time (on single core)
2. Both execute sequentially within themselves
3. Both can create children
4. If one is blocked, another can run

TYPES OF THREADS -- ULT vs KLT

Parameter

User Level Thread (ULT)
Kernel Level Thread (KLT)

Implemented by

User thread libraries
Operating System

OS awareness
OS does NOT know about threads
OS fully recognizes threads

Create/manage
Fast -- user space operations
Slow -- requires system calls

Context switch
No hardware support needed
Requires hardware support

Blocking
If 1 ULT blocks entire process blocked
If 1 KLT blocks other threads can continue

Multiprocessing
Multi-thread apps can't benefit from multiprocessing
True multiprocessing possible

Example
Java threads, POSIX threads
Windows threads, Solaris

ULT Advantages:

Simple and fast to create/switch
Can run on any OS (OS agnostic)
No Kernel mode privilege needed for switching

KLT Advantages:

True parallelism on multi-core processors
If one thread blocks, OS schedules another
Supports multithreading for Kernel routines

ULT Disadvantages:

Multi-thread app cannot benefit from multiprocessing
Single ULT blocking whole process halts

KLT Disadvantages:

Transferring control requires mode switch to Kernel
Slower to create and manage than ULT

3 MULTITHREADING MODELS

Model 1 -- Many-to-One

```

User threads: [T] [T] [T] [T] (many)

Kernel thread: [ K ] (one)

```

Characteristics:

- Many user threads mapped to one kernel thread
- Only one user thread can access kernel at a time
- True multiprocessing NOT possible (single kernel thread)
- If one user thread blocks all user threads blocked

Problem: Single kernel thread is a bottleneck.

Model 2 -- One-to-One

```

User threads: [T] [T] [T] (each user thread)

Kernel threads: [K] [K] [K] (has own kernel thread)

```

Characteristics:

- Each user thread gets its own kernel thread
 - True multiprocessing -- threads run on multiple processors
 - If one user thread blocks only that thread blocked
 - Problem: Creating too many kernel threads can degrade performance
- Example: Linux, Windows

Model 3 -- Many-to-Many (BEST)

User threads: [T] [T] [T] [T] [T] (many user threads)
 Kernel threads: [K] [K] [K] (<= user threads)

Characteristics:

- Many user threads same or lesser number of kernel threads
- Most flexible model
- Doesn't have the "too many kernel threads" problem of 1:1
- True parallelism possible
- If one user thread blocks OS can schedule other user threads

Why it's the BEST:

- Combines benefits of both Many-to-One and One-to-One
- System doesn't block if a particular thread is blocked
- Number of kernel threads is specific to the machine

THREAD vs PROCESS -- VISUAL SUMMARY

Process A	Process B
Code (shared)	Code (shared)
Data (shared)	Data (shared)
Files (shared)	Files (shared)
Thread 1 T2 T3	Thread 1 T2
[PC][Reg][Stk]	[PC][Reg][Stk]
isolated	isolated
Processes don't share memory	

Your 40-Second Script -- Threads

"A thread is the smallest unit of CPU execution within a process. Unlike processes, threads within the same process share code, data, and file resources -- they only have their own program counter, register set, and stack. This makes threads lightweight compared to processes. There are User Level Threads managed by user libraries -- fast but if one blocks the whole process stalls. And Kernel Level Threads managed by the OS -- slower but true parallelism is possible. The Many-to-Many model is best -- it maps multiple user threads to multiple kernel threads giving flexibility without bottlenecks."

Follow-Up Questions (Expect These)

Q: Why are threads called lightweight processes?

Because they share the parent process's code, data, and OS resources -- only program counter, registers, and stack are private. Creating a thread is much cheaper than creating a new process (no new memory space allocated).

Q: What is the benefit of multithreading in a web server?

A web server can handle multiple client requests simultaneously -- each request gets its own thread. Threads share the server's code and data (no duplication), and are created/destroyed quickly. Without threads, the server would handle one request at a time.

Q: Why can't ULT take advantage of multiprocessors?

The OS sees only the process, not individual user threads. So it schedules the process on one CPU at a time. Only kernel threads (visible to OS) can be scheduled on multiple CPUs simultaneously.

Q: What happens when a ULT makes a blocking system call?

The OS blocks the entire process -- because the OS doesn't know about user-level threads. All other threads in that process also stop. This is why KLT (or Many-to-Many model) is preferred for blocking I/O workloads.